M.Tech AI-ML · LCA1

Design and Deployment of an AI-Based Web Application for Classification using Neural Networks

Indian E-commerce Review Intelligence

Spam / fake-review detection · sentiment · topic dashboard

Deep-learning topics covered in depth: Cost functions in neural networks · Perceptron & back-propagation · Variants of gradient descent

Item	Details
Student name	[[Your name]]
Roll / enrolment no.	[[Your roll number]]
Programme / course	M.Tech AI-ML — LCA1
Assignment issued	8 September 2026 (deadline stated in brief: 20 September 2026)
Deployed application URL	[[Paste your Google Cloud Run service URL here]]
Source code	ecom_review_intelligence.zip (submitted with this report)

Contents

1 Introduction and objectives
2 Dataset, pre-processing and feature extraction
3 The Perceptron — working and limitations
4 Multi-layer perceptron and forward propagation
5 Cost (error) functions
6 Back-propagation and convergence
7 Variants of gradient descent
8 Hyper-parameter tuning
9 Performance evaluation
10 Web application
11 Google Cloud deployment
12 Limitations and future work
13 Conclusion
References
Appendix A How to run the project step by step
Appendix B Submission checklist

How to read this report. Every number, table and figure is generated by the code in the accompanying zip (`run_pipeline.py` → `outputs/results.json`) and inserted automatically by `report/build_report.py`; re-running the pipeline and rebuilding the report keeps the two consistent. Yellow-highlighted items are things only the student can supply (name, deployed URL, screenshots).

1 Introduction and objectives

Online marketplaces such as Flipkart, Amazon.in, Myntra, Meesho and Nykaa rely on customer reviews to drive purchase decisions. A growing share of those reviews is not genuine: promotional links, paid '5-star' reviews, copy-pasted praise and outright junk. This project builds an **AI-based web application** that takes a review typed by a user and answers, in real time, **Spam / Not Spam**. For genuine reviews it additionally reports **sentiment** (Negative / Neutral / Positive), **topics** (delivery, packaging, quality, price, battery, fit …) and feeds a **review-intelligence dashboard** (trend, sentiment split, rating distribution, top positive / negative topics, price-vs-rating analysis).

The assignment brief asks students to demonstrate activation functions, a cost function, back-propagation and hyper-parameter tuning with a multi-layer perceptron, and then deploy on Google Cloud. In addition, three deep-learning topics are treated in depth and implemented **from scratch in NumPy** (no TensorFlow / PyTorch), so that every equation in the report corresponds to a few lines of visible code:

- **Cost functions in neural networks** — MSE, binary cross-entropy, categorical cross-entropy; their gradients and behaviour (Section 5).
- **Perceptron and back-propagation** — Rosenblatt perceptron, its XOR limitation, the MLP, chain-rule back-propagation and a numerical gradient check (Sections 3, 4, 6).
- **Variants of gradient descent** — Batch, Stochastic, Mini-batch, Momentum, Nesterov, AdaGrad, RMSProp, Adam (Section 7).

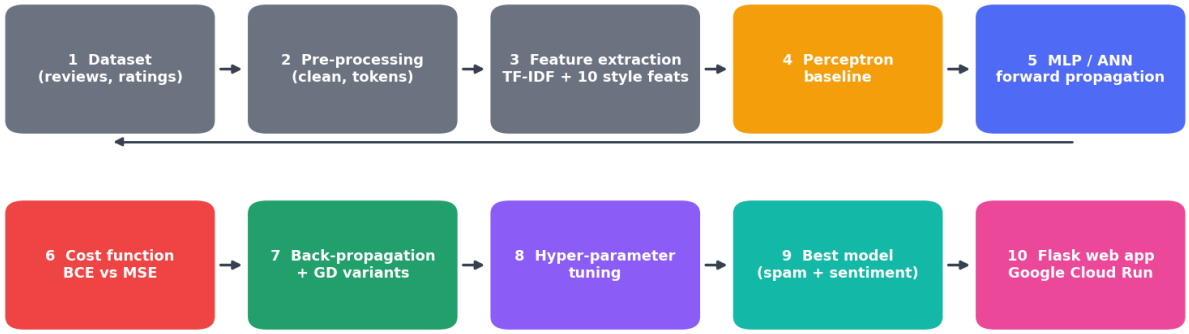


Figure 1 — End-to-end architecture, following the suggested architecture in the brief.

Requirement coverage.

Brief requirement	Where addressed
1 Dataset preparation, pre-processing, feature extraction	Section 2
2 Multi-layer perceptron: working and limitations	Sections 3, 4
3 Neural network ≥ 1 hidden layer, forward propagation, ReLU / Sigmoid / …	Section 4
4 Cost function implemented, explained, loss-vs-epochs plot	Sections 5, 6
5 Back-propagation and demonstration of convergence	Section 6
6 Hyper-parameter tuning (≥ 3 hyper-parameters)	Section 8 (six studied)
7 Accuracy, precision, recall, F1, confusion matrix	Section 9
8 Web interface: Input message → AI model → Spam / Not Spam	Section 10
9 Google Cloud deployment	Section 11

2 Dataset, pre-processing and feature extraction

2.1 Dataset

Important note on data. The development environment had no internet access, so a public review corpus could not be downloaded. Instead the project includes a documented generator (`reviewintel/synth.py`)

that produces a **synthetic but realistic Indian e-commerce review dataset**: English and Hinglish text (e.g. 'Packing bahut kharab thi'), six categories, 120 products with INR prices, platforms (Flipkart, Amazon.in, Myntra, Meesho, Nykaa, JioMart), 1–5 star ratings, dates over 2026 with growing volume, and a spam label. Seven spam / fake types are simulated: promotional links, paid-review solicitations, gibberish, off-topic posts, generic fake praise, genuine-looking review + promo tail, and *subtle fakes* with no link at all. To keep the task realistic, genuine 'hard negatives' (genuine reviews quoting a customer-care number, '5 stars', WhatsApp …) and 1.5 % annotation noise are included. **Results therefore describe this dataset and should not be read as accuracy on real marketplace traffic.** A real CSV can be plugged in with `python run_pipeline.py --data my_reviews.csv` (Appendix A).

Property	Value
Source	synthetic generator (n=8000, seed=42)
Reviews	8,000
Products / categories	120 / 6
Spam (positive class) share	20.7%
Average review length	86.2 characters
Train / validation / test (stratified, seed 42)	5,600 / 1,200 / 1,200

2.2 Pre-processing

- **Normalisation:** lower-casing; elongated characters squeezed ('soooo' → 'soo'); emoji and punctuation removed; '90%' → '90 percent'.
- **Spam-aware tokens:** URLs, Indian mobile numbers and @handles are *replaced by the tokens* `urltoken`, `phonetoken`, `handletoken` instead of being deleted, because their presence is itself a signal.
- **No stop-word removal:** words such as 'not' and 'no' carry sentiment, and Hinglish function words ('hai', 'ekdum') are not in standard stop-word lists.
- **Split before fitting:** vocabulary, IDF and scaling statistics are learnt on the training split only (no leakage into validation / test).

2.3 Feature extraction

Each review becomes a vector of **1,510 numbers** = 1,500 TF-IDF features + 10 style features:

- **TF-IDF** over unigrams and bigrams (vocabulary of the 1,500 most frequent n-grams with document frequency ≥ 3), implemented from scratch: weight = (1 + ln tf) · idf with idf = ln((1+n)/(1+df)) + 1, L2-normalised per review.
- **Ten hand-crafted 'style' features** that capture *how* a review is written: log length in characters and words, upper-case ratio, digit ratio, number of exclamation marks, contains-URL, contains-phone, repeated-word ratio, special-character ratio and mean word length — standardised with training statistics.

The extractor is saved as JSON, so the deployed web app needs only NumPy and Flask.

3 The Perceptron — working and limitations

The perceptron (Rosenblatt, 1958) is a single artificial neuron: it forms a weighted sum of the inputs, adds a bias and applies a hard threshold.

$$z = w \cdot x + b \qquad \hat{y} = 1 \text{ if } z > 0, \text{ else } 0$$

$$\text{if the sample is misclassified } (y \in \{-1,+1\}): \quad w \leftarrow w + \eta \cdot y \cdot x\,, \quad b \leftarrow b + \eta \cdot y$$

Learning is therefore **error-driven**: weights change only when a prediction is wrong. The perceptron convergence theorem guarantees that this rule reaches zero training errors in a finite number of updates **if and only if the classes are linearly separable**.

Result on review data. Trained for 15 epochs on the 1,510-dimensional features, the perceptron makes 645 mistakes in epoch 1 and still 217 in epoch 15: it never reaches zero because spam and genuine reviews are *not perfectly* linearly separable (subtle fakes look like genuine praise). On the test set it scores accuracy 97.25%, precision 98.21%, recall 88.35%, F1 93.02%. This is a strong baseline because most spam types (links, phone numbers, junk) *are* separable by a single hyper-plane.

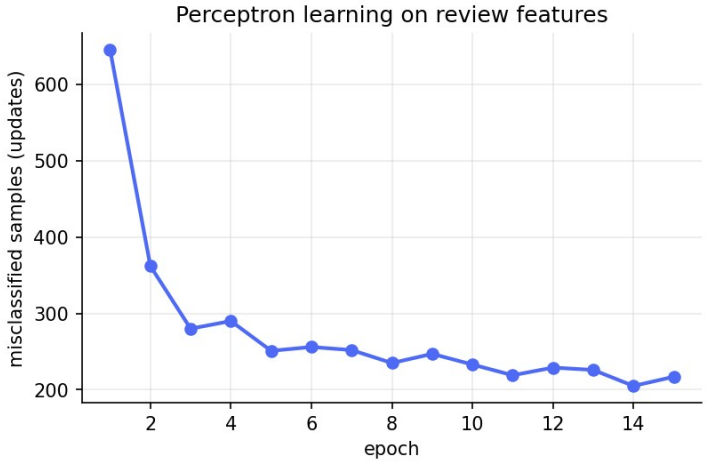


Figure 2 — Perceptron updates per epoch: errors fall but never reach zero.

3.1 Limitations — the XOR problem

A single perceptron can only draw **one straight decision boundary**. The classic counter-example is XOR, whose two classes cannot be separated by a line. Our demonstration gives the perceptron 50% accuracy (chance level) while a small MLP with one hidden layer solves it with 100% (Figure 3). Hidden layers with non-linear activations let the network build *curved / piece-wise* boundaries — the motivation for the multi-layer perceptron (Minsky & Papert, 1969; Rumelhart et al., 1986).

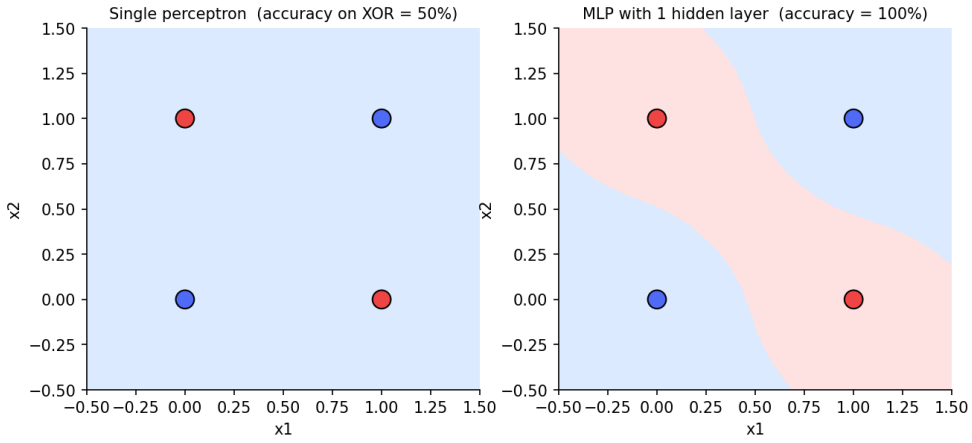


Figure 3 — XOR: a single perceptron (left) cannot separate the classes; an MLP with one hidden layer (right) can.

4 Multi-layer perceptron and forward propagation

A multi-layer perceptron stacks layers of neurons. Layer *l* computes a linear transformation followed by a non-linear activation *f*:

$$z^{(l)} = a^{(l-1)} W^{(l)} + b^{(l)} \qquad a^{(l)} = f(z^{(l)}), \qquad a^{(0)} = x$$

The deployed spam network is `1,510 → 4 → 1` with **RELU** hidden units and a **sigmoid** output that is read as P(spam); it has 6,049 trainable parameters. Forward propagation is the function `MLP.forward` in `reviewintel/nn.py`. Weights use He initialisation (ReLU) or Xavier / Glorot initialisation (sigmoid / tanh) to keep activations well scaled.

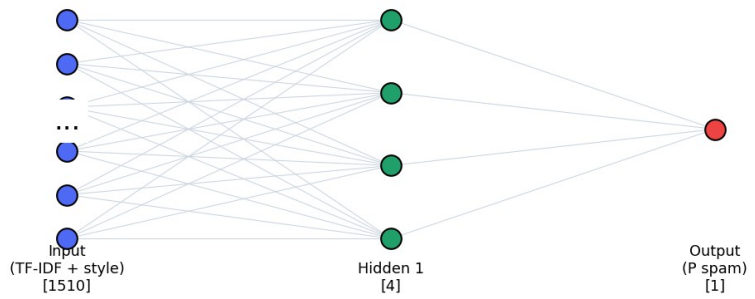


Figure 4 — Architecture of the deployed network (only a few neurons per layer drawn).

4.1 Activation functions

Activation	f(z)	f′(z)	Remarks
ReLU	max(0, z)	1 if z>0 else 0	Cheap, no saturation for z>0; 'dead neuron' risk
Leaky ReLU	z if z>0 else 0.01z	1 or 0.01	Avoids dead neurons
Sigmoid	$1/(1+e^{-z})$	a(1−a)	Output in (0,1) → probability; saturates → vanishing gradients in hidden layers
Tanh	$(e^{z}-e^{-z})/(e^{z}+e^{-z})$	$1-a^2$	Zero-centred; still saturates
Softmax	$e^{zk} / \Sigma e^{zj}$	(with cross-entropy: a−y)	Used for the 3-class sentiment output

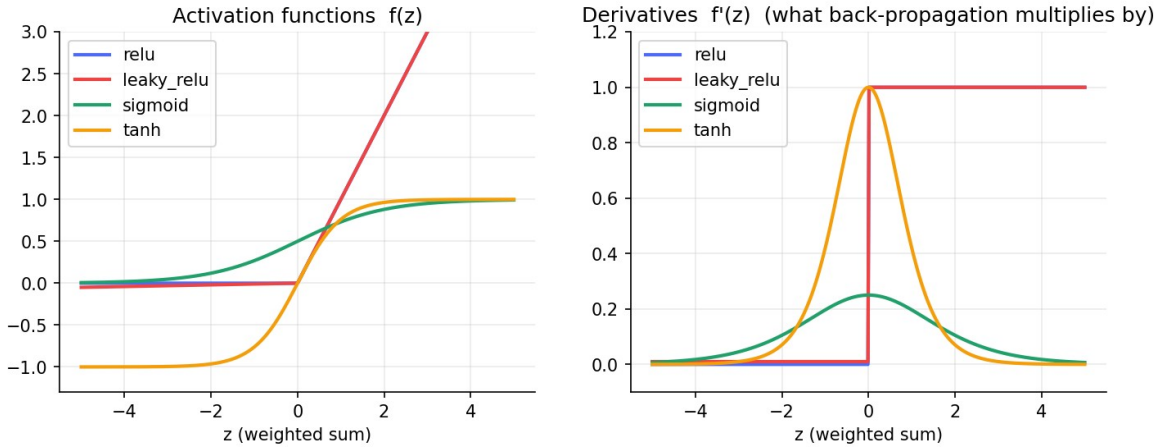


Figure 5 — Activation functions (left) and the derivatives that back-propagation multiplies by (right).

4.2 Working and limitations of the MLP

- **Working:** hidden layers learn intermediate feature combinations (e.g. 'link + urgency words') and the output layer converts them into a probability; the universal-approximation theorem says one wide hidden layer can approximate any continuous function.
- **Limitations:** (i) needs many labelled examples and is easily over-fitted — see Section 8; (ii) the *bag-of-n-grams* input throws away word order and context, so sarcasm or subtle fake praise is hard; (iii) many hyper-parameters; (iv) non-convex cost surface, so training may stall in poor regions; (v) it is a 'black box' — we add an occlusion explanation in the web app to partly compensate; (vi) it cannot handle unseen vocabulary or spelling variants beyond the training vocabulary (typical for Hinglish).

5 Cost (error) functions

A cost function J turns the network's mistakes into a single number that training minimises. For m samples with target y and network output a:

Cost	Formula	Used for	Output delta ∂J/∂z (starts back-prop)
Mean squared error (MSE)	$J = (1/m)\ \Sigma\ (a - y)^2$	regression; (here: comparison)	2(a−y)·a(1−a)/m (with sigmoid)
Binary cross-entropy (BCE)	$J = -(1/m)\ \Sigma\ [\ y \ln a + (1-y) \ln(1-a)\ ]$	binary classification → spam head	(a − y)/m
Categorical cross-entropy	$J = -(1/m)\ \Sigma_i\ \Sigma_k\ y_{ik} \ln a_{ik}$	multi-class → sentiment head	(a − y)/m (with softmax)

Why cross-entropy for classification? BCE is the negative log-likelihood of a Bernoulli model, so it penalises a *confidently wrong* prediction very heavily (cost → ∞ as a → 1−y), and — crucially — the sigmoid's derivative cancels in the gradient, leaving the simple error (a − y). With MSE the gradient contains the factor a(1−a), which is almost zero when the neuron is saturated, i.e. exactly when it is confidently wrong (Figure 6, middle).

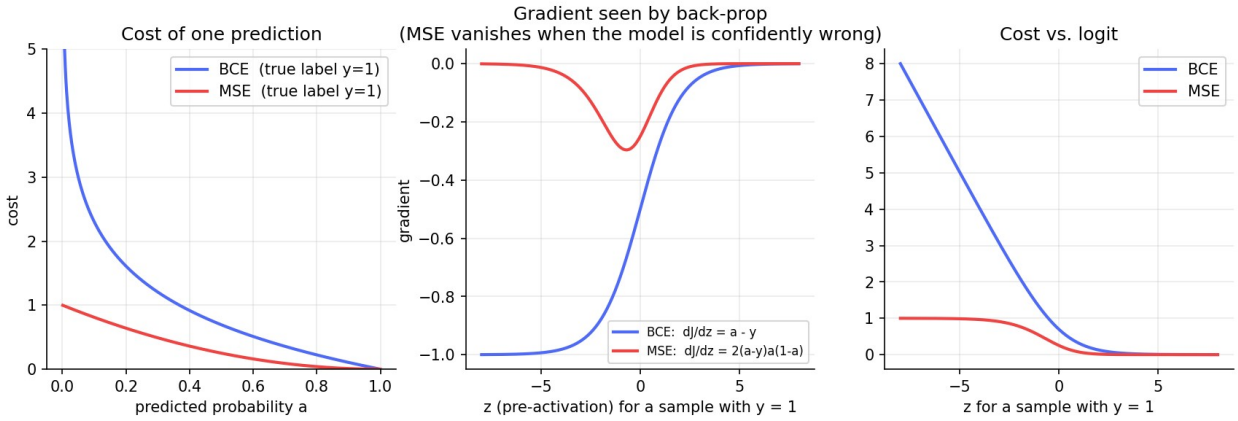


Figure 6 — Left: cost of one prediction; middle: gradient reaching back-propagation (MSE vanishes for confident errors); right: cost against the logit.

5.1 Experiment: BCE vs MSE

The same network (1,510→32→1, ReLU, mini-batch 64, 20 epochs) was trained with each cost, using Adam and plain gradient descent. For plain GD each cost received its own best learning rate from a small grid. A further pair of runs starts from a **'confidently wrong'** state (output bias set to −8, so the net initially predicts 'genuine' for everything) with the *same* learning rate (0.5) for both costs. Note that costs of different functions are on different scales, so compare accuracy / F1, not the raw cost values.

Set-up	lr	final train cost	val accuracy	val F1
BCE + Adam	0.001	0.0520	97.83%	0.9458
MSE + Adam	0.001	0.0136	97.58%	0.9400
BCE + GD	0.2	0.0847	97.67%	0.9414
MSE + GD	0.5	0.0207	97.67%	0.9421
BCE + GD (confidently-wrong start)	0.5	0.0742	97.00%	0.9253
MSE + GD (confidently-wrong start)	0.5	0.1983	79.42%	0.0159

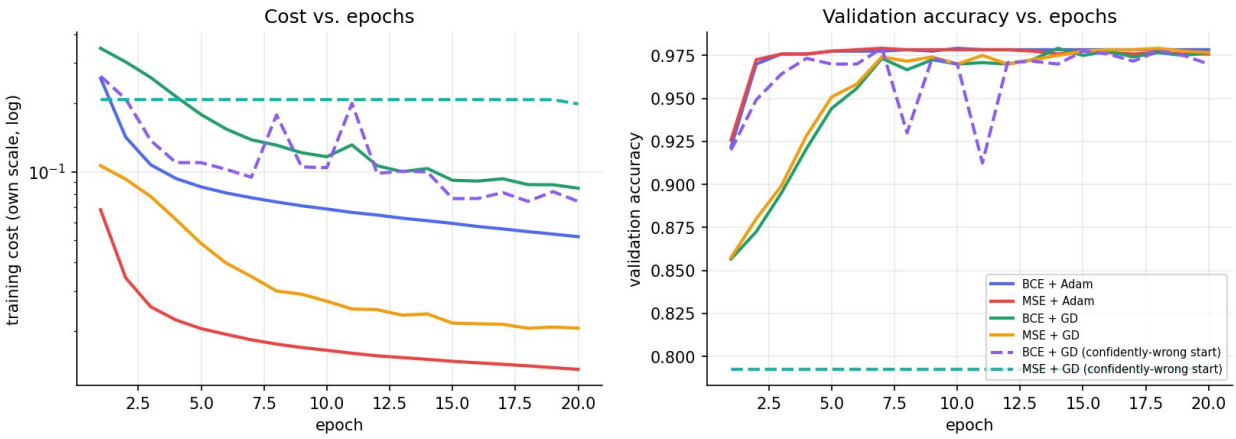


Figure 7 — Training cost (log scale) and validation accuracy for each cost / optimiser combination. Dashed = confidently-wrong start.

Findings.

- From a normal start the two costs end up **equally good** on this easy problem (Adam: F1 0.946 vs 0.940; plain GD: 0.941 vs 0.942); adaptive optimisers largely hide the gradient-scaling difference. We do not claim that BCE is more accurate here.
- From the confidently-wrong start the difference is dramatic: with BCE the network recovers to 97.0% validation accuracy (F1 0.925), whereas with MSE it is **stuck at 79.4% (F1 0.016)** — it keeps predicting 'genuine' for every review because the gradient 2(a−y)a(1−a) is almost zero. This is the practical reason cross-entropy is preferred for sigmoid / softmax outputs.
- BCE also yields better-calibrated probabilities, which the web app shows as P(spam).

6 Back-propagation and convergence

Back-propagation computes ∂J/∂W for **every** weight in one backward sweep by applying the chain rule layer by layer. With $\delta^{(l)} = \partial J/\partial z^{(l)}$:

$$\delta^{(L)} = \partial J/\partial z^{(L)} = (a^{(L)} - y)/m \qquad (BCE + sigmoid,\ output\ layer)$$

$$\delta^{(l)} = (\delta^{(l+1)} \cdot W^{(l+1)T}) \odot f'(z^{(l)}) \qquad (propagate\ the\ error\ backwards)$$

$$\partial J/\partial W^{(l)} = a^{(l-1)T} \cdot \delta^{(l)} \qquad \partial J/\partial b^{(l)} = \Sigma\ rows\ of\ \delta^{(l)}$$

$$W^{(l)} \leftarrow W^{(l)} - \eta \cdot \partial J/\partial W^{(l)} \qquad (gradient\text{-}descent\ step;\ Section\ 7\ gives\ the\ variants)$$

Algorithm per mini-batch: (1) forward propagate and cache every z and a; (2) compute the output delta; (3) loop from the last layer to the first, computing weight gradients and passing the error back through the transpose of the weights and the activation derivative; (4) let the optimiser update all parameters. This is implemented in `MLP.backward` (≈ 10 lines).

6.1 Correctness: numerical gradient check

To prove the hand-written back-propagation is correct, its analytic gradient is compared with the centred finite-difference gradient (J(θ+ε) − J(θ−ε))/2ε for every parameter of a small network. A relative error below 1e-7 indicates correct gradients.

Hidden act. + cost + output	Relative error
tanh+bce+sigmoid	8.55e-11
relu+bce+sigmoid	3.82e-11
sigmoid+mse+sigmoid	7.35e-11
leaky_relu+mse+linear	1.20e-11
relu+cce+softmax	4.55e-11

All combinations agree to ≈ 1e-10, so the same code path can be trusted for the spam and sentiment networks (it is also part of the unit tests).

6.2 Demonstration of convergence

The final spam network was trained with mini-batch Adam and early stopping (patience 8 on validation cost). Training cost fell from **0.589** (epoch 1) to **0.074**, while the validation cost reached its minimum of **0.106** at epoch **29**; training stopped at epoch 37 and the weights of the best epoch were restored. The norm of the full-batch gradient shrank from **0.352** to **0.0089** (39× smaller) — the gradient heading towards zero is the signature of convergence to a (local) minimum. Training and validation accuracy rise together and level off near 97.8%, i.e. no serious over-fitting.

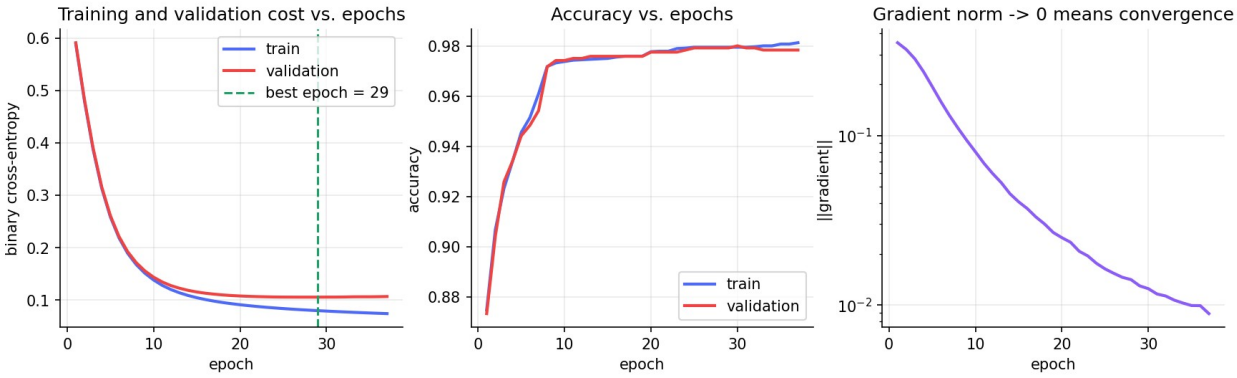


Figure 8 — Training / validation cost vs epochs (left), accuracy (middle) and gradient norm (right) of the deployed model.

7 Variants of gradient descent

All variants follow the idea $\theta \leftarrow \theta - (\text{step})$, differing in **how much data** is used for each gradient (Batch / Stochastic / Mini-batch) and **how the step is built** from the gradient history (Momentum … Adam). In the notation $g = \partial J/\partial\theta$, learning rate η:

Variant	Update rule	Idea / trade-off
Batch GD	$\theta \leftarrow \theta - \eta \cdot g$ (g over all m samples)	Exact gradient, but only 1 update per epoch → slow; needs all data in memory
Stochastic GD	same, g from 1 sample	Very frequent, noisy updates; escapes shallow minima; slow in Python (no vectorisation)
Mini-batch GD	same, g from 32–256 samples	Best compromise: vectorised, moderate noise — the practical default
Momentum	$v \leftarrow \mu v - \eta g$; $\theta \leftarrow \theta + v$	Accumulates velocity → damps zig-zag in ravines, accelerates along consistent directions
Nesterov (NAG)	$v \leftarrow \mu v - \eta g$; $\theta \leftarrow \theta - \mu v_{prev} + (1+\mu)v$	Look-ahead version of momentum; corrects overshoot earlier
AdaGrad	$G \leftarrow G + g^2$; $\theta \leftarrow \theta - \eta g/(\sqrt{G}+\varepsilon)$	Per-parameter step: large for rare features (good for sparse TF-IDF); step decays → may stall
RMSProp	$s \leftarrow \rho s + (1-\rho)g^2$; $\theta \leftarrow \theta - \eta g/(\sqrt{s}+\varepsilon)$	Exponential average fixes AdaGrad's decaying step
Adam	m, v moving averages of g, g^2 (bias-corrected); $\theta \leftarrow \theta - \eta \cdot \hat{m}/(\sqrt{\hat{v}}+\varepsilon)$	Momentum + RMSProp; robust default, little tuning

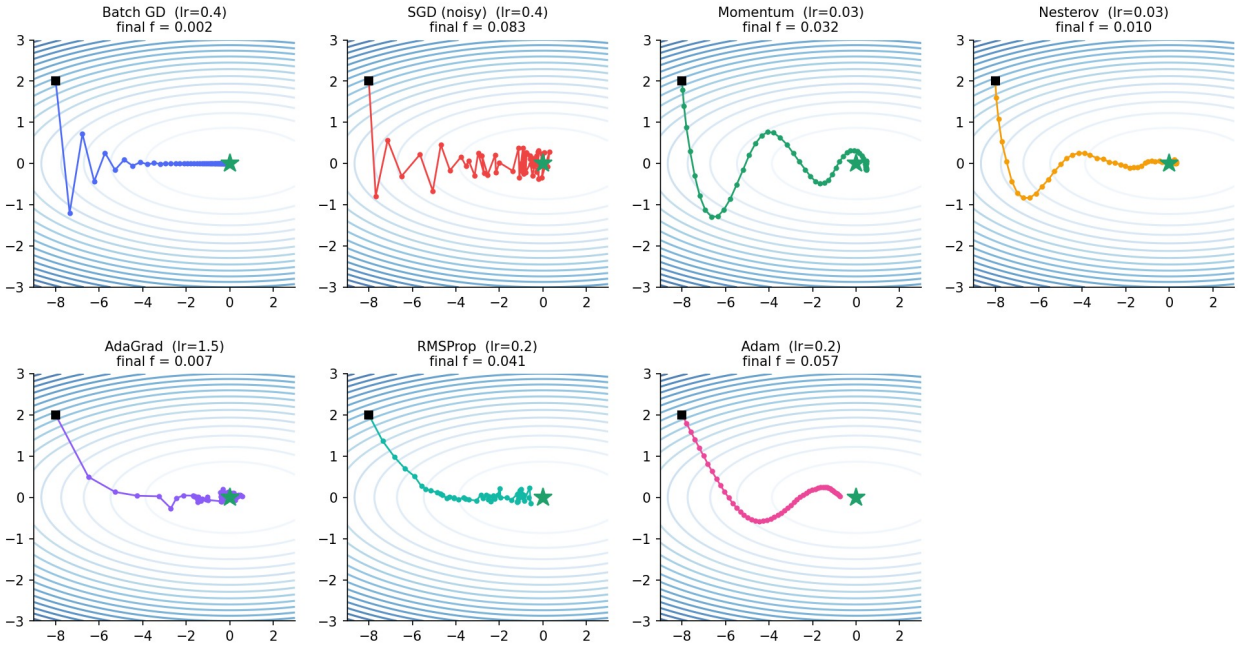


Figure 9 — Toy illustration on an elongated bowl $f = 0.1x^2 + 2y^2$ (start ■, minimum ★). Gradients of all but 'Batch GD' carry artificial noise to mimic mini-batches. Batch GD zig-zags across the steep direction; momentum overshoots then spirals in; Nesterov corrects earlier; AdaGrad / RMSProp rescale each axis; Adam follows a smooth curve.

7.1 Experiment on the review data

Each variant trained the baseline network for 15 epochs. Because the best learning rate differs wildly between variants, each received a small grid of learning rates and the one giving the **lowest final training cost** was kept (an optimiser is judged on how well it minimises the cost in a fixed budget). 'Updates' is the number of weight updates performed.

Variant	batch	best lr	updates	final train cost	val F1	time (s)
Batch GD	full	5	15	0.5224	0.271	0.66
Stochastic GD (bs=1)	1	0.05	84,000	0.0535	0.950	14.38
Mini-batch GD (bs=64)	64	1.0	1,320	0.0666	0.936	0.69
Momentum	64	0.1	1,320	0.0622	0.941	0.87
Nesterov (NAG)	64	0.3	1,320	0.0524	0.926	0.67
AdaGrad	64	0.3	1,320	0.0200	0.932	0.9
RMSProp	64	0.01	1,320	0.0587	0.944	0.78
Adam	64	0.03	1,320	0.0199	0.928	1.18

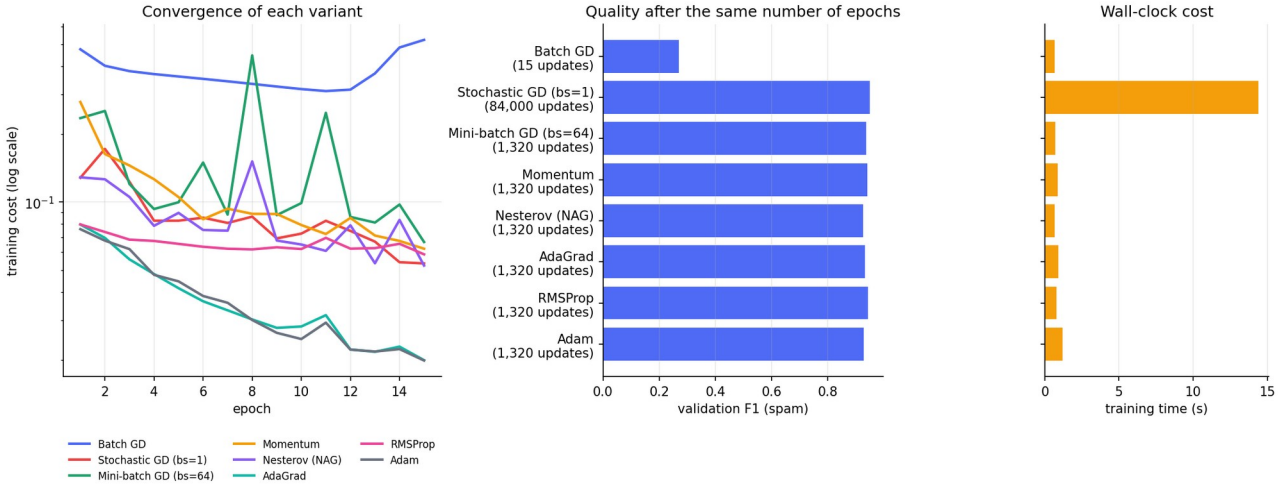


Figure 10 — Convergence (left), validation F1 (middle) and wall-clock time (right) of the eight variants.

Findings.

- **Batch GD** performs only 15 updates in 15 epochs and is badly under-trained (val F1 0.27); it would need hundreds of epochs.
- **Stochastic GD** needs 84,000 updates and 14.4 s — about 21× the time of mini-batch — although it reaches a very good model (val F1 0.950). The noisy updates act as regularisation but the cost is wall-clock time.
- **Mini-batch GD** delivers nearly the same quality in 1,320 updates and 0.7 s: the practical default.
- **Adaptive methods:** Adam and AdaGrad reach the lowest training cost (0.020 and 0.020) — sparse TF-IDF features benefit from per-parameter step sizes. Momentum / Nesterov / RMSProp are close behind.
- Lowest training cost is **not** the same as best validation F1: the differences between the mini-batch optimisers in F1 (about 1–2 points on only 1,200 validation reviews) are within sampling noise, so we do not rank them by F1. We use Adam in the final model because it is robust to the learning rate and needs little tuning.

8 Hyper-parameter tuning

Six hyper-parameters were studied **one at a time around the baseline** (hidden = [32], ReLU, BCE, Adam, lr = 1e-3, batch 64, 20 epochs). Every setting was trained with several random seeds and judged on **validation F1 of the spam class** (the test set was never used for selection). Error bars show ± 1 standard deviation across seeds.

Hyper-parameter	Validation F1 for each value	Best
Learning rate	0.0001: 0.922 0.0003: 0.943 0.001: 0.944 0.003: 0.937 0.01: 0.929 0.03: 0.927	0.001
Hidden neurons	4: 0.949 8: 0.947 16: 0.945 32: 0.944 64: 0.944 128: 0.941 256: 0.940	4
Hidden layers (32 neurons each)	1: 0.944 2: 0.935 3: 0.933 4: 0.935	1
Batch size	8: 0.937 16: 0.936 32: 0.942 64: 0.944 128: 0.946 256: 0.944 512: 0.939	128
Hidden activation	relu: 0.944 leaky_relu: 0.944 tanh: 0.940 sigmoid: 0.939	relu
Number of epochs	2: 0.919 5: 0.943 10: 0.947 20: 0.944 40: 0.943 60: 0.936	10

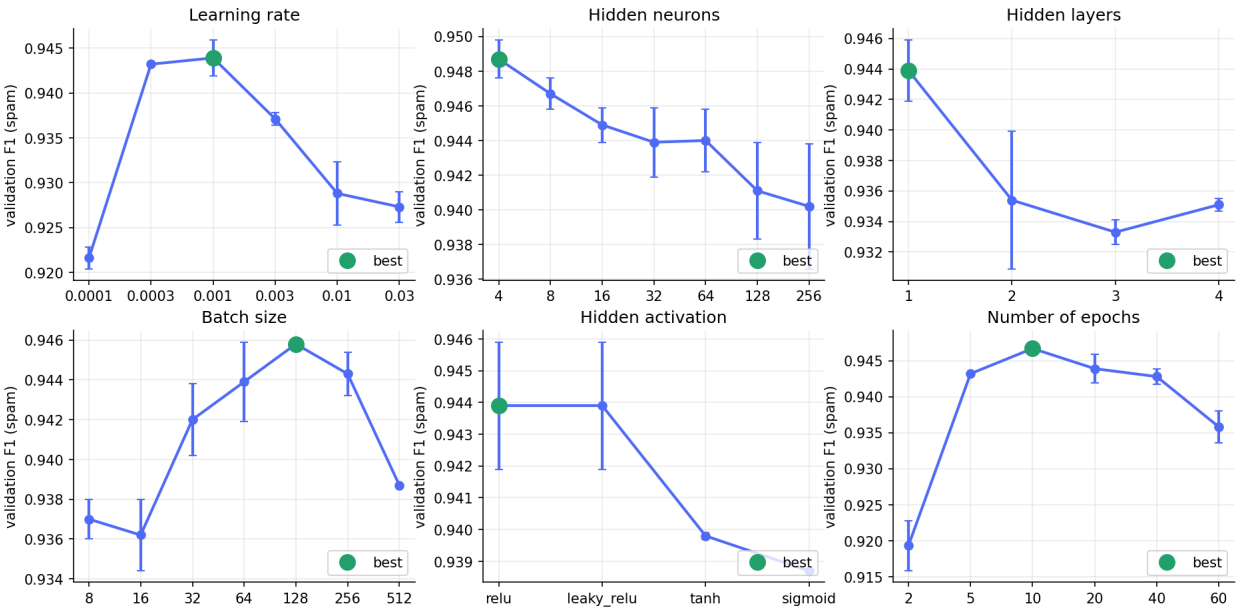


Figure 11 — Validation F1 against each hyper-parameter (green = best value).

Observations.

- **Learning rate:** too small (0.0001) under-fits within 20 epochs (F1 0.922); the best value is 0.001; large rates (0.03) make updates overshoot (F1 0.927).

- **Hidden neurons / layers:** the problem is close to linearly separable, so more capacity does not help — F1 decreases from 0.949 with 4 neurons to 0.940 with 256, and a single hidden layer is best (extra layers lose ≈ 1 point). Larger networks fit the training noise.
- **Batch size:** best around 128; very small batches give noisy gradients and very large ones make too few updates in the epoch budget.
- **Activation:** ReLU and Leaky-ReLU (0.944) beat tanh (0.940) and sigmoid (0.939) — saturating hidden activations learn more slowly.
- **Epochs:** 2 epochs under-fit (F1 0.919), the best is 10, and 60 epochs over-fit (F1 0.936) — Figure 12; early stopping automates this choice.
- Most differences are only 0.5 – 1.5 F1 points; with 1,200 validation reviews these are close to the seed-to-seed noise, except for the extremes (very low learning rate, very few epochs), which are clearly worse.

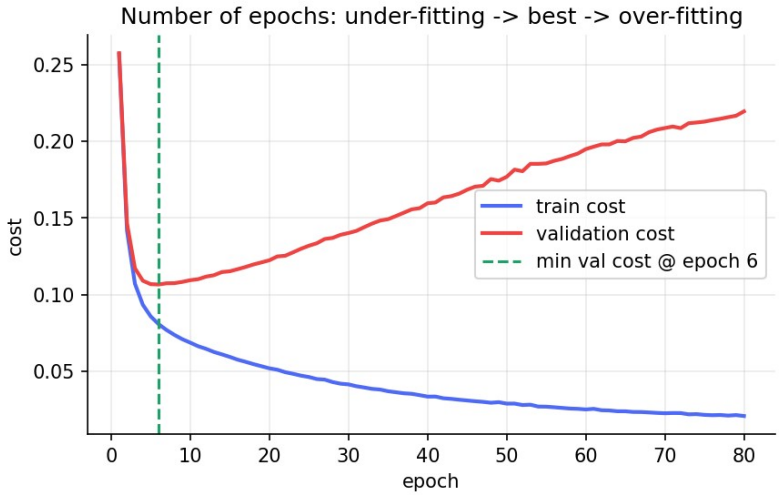


Figure 12 — Long run: training cost keeps falling while validation cost turns upward (over-fitting).

8.1 Selecting the final configuration

The best value of each hyper-parameter was combined and compared with the baseline (and a variant with L2 regularisation) using several seeds with early stopping:

Candidate	hidden	lr	batch	activation	L2	val F1 (mean ± sd)
baseline	[32]	0.001	64	relu	0	0.9432 ± 0.0000
combined best	[4]	0.001	128	relu	0	0.9476 ± 0.0000
combined + L2	[4]	0.001	128	relu	0.0001	0.9458 ± 0.0000

Selected: **combined best** — hidden layer [4], RELU, Adam lr 0.001, batch 128, BCE, up to 60 epochs with early stopping. It has only 6,049 parameters versus 48,385 for the baseline.

9 Performance evaluation

All numbers in this section are on the **held-out test set** (1,200 reviews), with 'spam' as the positive class. Decision threshold: 0.5 (a threshold sweep on the validation set did not improve F1, so the default 0.5 is kept).

Model	Accuracy	Precision	Recall	F1-score	AUC
Perceptron (baseline)	97.25%	98.21%	88.35%	93.02%	—
MLP baseline (untuned, 32 hidden)	97.83%	99.12%	90.36%	94.54%	0.977
Tuned MLP (deployed)	**97.75%**	**99.55%**	**89.56%**	**94.29%**	**0.978**

Over five random initialisations the tuned model scores F1 94.33% ± 0.16% and accuracy 97.77% ± 0.06% (mean ± standard deviation), so the result is stable. The deployed model is the seed with the best *validation* F1.

$$Accuracy = (TP+TN)/N \qquad Precision = TP/(TP+FP) \qquad Recall = TP/(TP+FN) \qquad F1 = 2{\cdot}P{\cdot}R/(P+R)$$

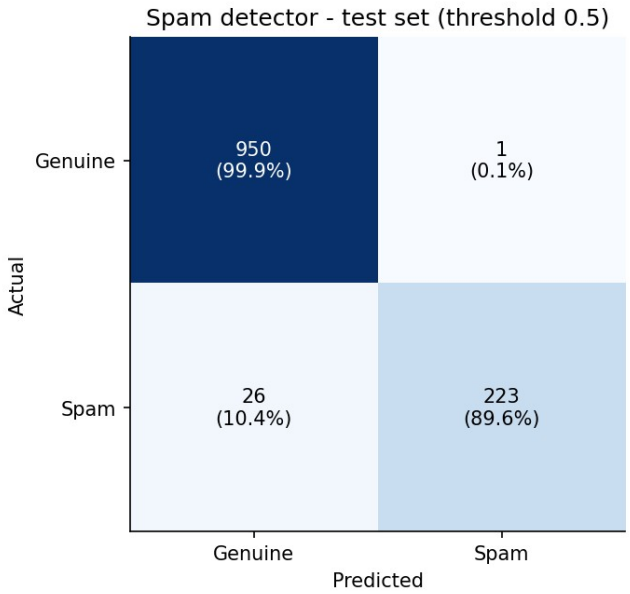


Figure 13 — Confusion matrix on the test set: TN = 950, FP = 1, FN = 26, TP = 223.

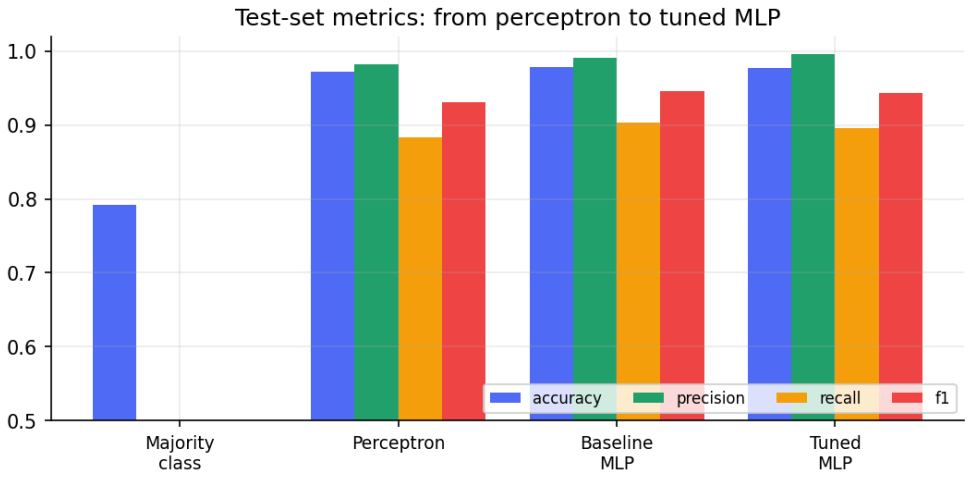


Figure 14 — Test metrics from the majority-class rule to the perceptron, baseline MLP and tuned MLP.

9.1 Interpretation

- **Precision 99.55%** — only 1 genuine review out of 951 was wrongly flagged: legitimate customers are almost never blocked, which matters most for a marketplace.
- **Recall 89.56%** — 26 of 249 spam reviews were missed. Recall by spam type shows where: generic fake praise 100%, gibberish 95%, hybrid 100%, offtopic 100%, paid review 100%, promo url 100%, subtle fake 82%. Link / phone / paid-review / junk spam is caught almost perfectly; the misses are **subtle fakes** that read like genuine praise and a few reviews whose label was deliberately flipped as annotation noise.
- **Perceptron → MLP:** the non-linear hidden layer adds about 1.5 F1 points over the perceptron (ROC-AUC of the MLP: 0.977).
- **Tuning:** tuning did *not* improve test accuracy over the untuned baseline (F1 94.29% vs 94.54%; the difference is within seed noise). Its benefit is a **8× smaller network** (6,049 vs 48,385 parameters) with equal quality, and evidence that the baseline was already close to the ceiling of this feature set.

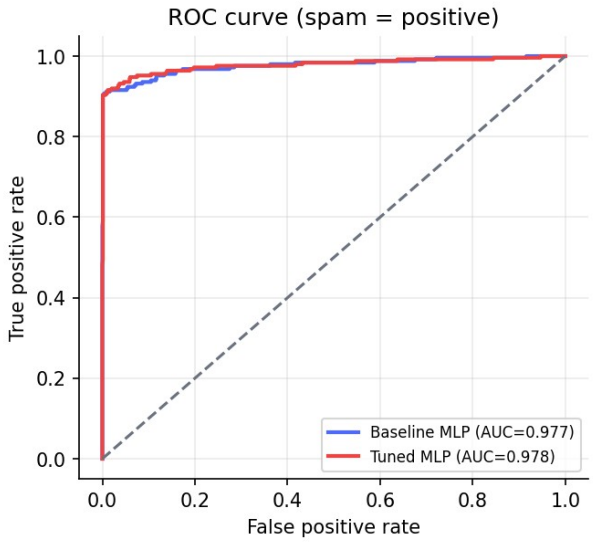


Figure 15 — ROC curve of the baseline and tuned MLP.

9.2 Error analysis

Missed spam review (false negative)	P(spam)	Comment
Waste of money. Looks nothing like the pictures, poor material. Avoid.	0.004	label-noise sample (genuine text labelled spam)
Display: awesome. Bas honestly, the camera is superb. Will buy again.	0.006	label-noise sample (genuine text labelled spam)
Could be better. Performance is snappy and apps open instantly. Overpriced for what you actuall	0.010	label-noise sample (genuine text labelled spam)
Totally satisfied! Quantity: awesome. Thank you seller.	0.012	label-noise sample (genuine text labelled spam)
Totally satisfied Customer care resolved my issue in a single call But honestly the battery bac	0.015	label-noise sample (genuine text labelled spam)

Genuine review flagged as spam (false positive)	P(spam)
Very good product very good seller very good delivery very good quality very good	0.982

Most remaining errors are intrinsically ambiguous (indistinguishable from real reviews or mislabelled), so further gains would need richer signals such as reviewer history, purchase verification, timing bursts or a sequence model — see Section 12.

9.3 Sentiment head (extension)

A second network with the same hidden layer, a **3-way softmax** output and **categorical cross-entropy** is trained on genuine reviews only (labels from the star rating: ≥4 Positive, 3 Neutral, ≤2 Negative). Test accuracy 87.07%, macro-F1 0.828; per-class F1: Negative 0.872, Neutral 0.685, Positive 0.925. The Neutral class is hardest (mixed opinions such as ‘fast delivery but average battery’), and 5 % of ratings were simulated as mis-rated, which caps accuracy; making the network wider (16 – 64 neurons, or two layers) gave no improvement (≈ 87 %), confirming the limit comes from the labels, not the capacity.

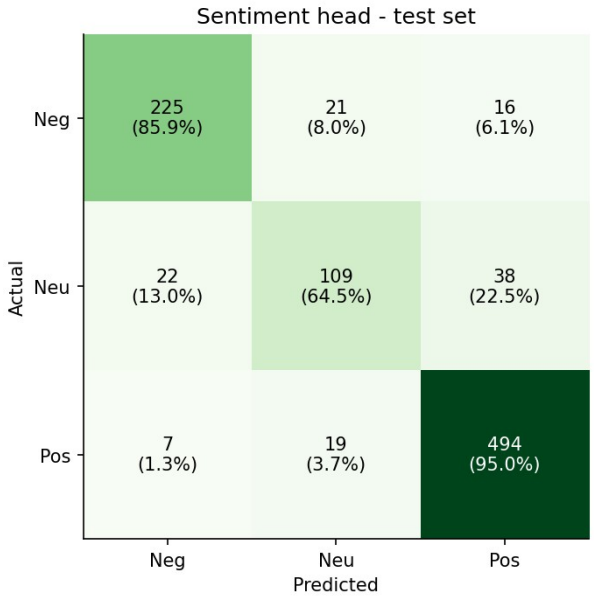


Figure 16 — Sentiment confusion matrix (test set).

10 Web application

The web interface is a small **Flask** application (`webapp/app.py`). It loads the saved weights with NumPy only and answers within milliseconds on a single CPU.

Input message → AI model → Spam / Not Spam

Page / endpoint	Purpose
/ (Analyzer)	Text box + example buttons. Shows verdict (Spam / Not Spam), P(spam) with threshold marker, sentiment bars, topics, rule-based spam signals (link, phone, repeated words …) and the words that most influenced the decision (occlusion: remove a word, see how P(spam) changes).
/dashboard	Review-intelligence dashboard: KPI cards (products, categories, reviews, % spam, average rating), review trend, sentiment breakdown, rating distribution, top-5 positive / negative topics, price-vs-rating scatter, spam by category, common keywords.
/model	Metrics and all experiment figures (confusion matrix, ROC, cost, gradient-descent and hyper-parameter studies).
POST /api/predict	JSON API: {"text": "…"} → label, probability, sentiment, topics, signals.
GET /health	Liveness check used by Cloud Run.

10.1 Example predictions from the running application

Input message	Prediction	P(spam)	Sentiment
Delivery was super fast and the battery easily lasts a full day. Camera is sharp too. To…	Not Spam	0.041	Positive
Very disappointed. The box was torn and the fabric is rough. Refund still not received a…	Not Spam	0.008	Negative
Packing bahut kharab thi aur delivery late ho gayi, quality bhi bekaar hai.	Not Spam	0.048	Negative
Get 90% OFF on all mobiles today only! Visit www.dealzmart-deals.in or WhatsApp 98765012…	Spam	1.000	n/a (spam)
Seller is giving Rs 100 refund for a 5 star review. DM me for details.	Spam	0.977	n/a (spam)
Best product ever!!! Must buy!!! 5 star 5 star 5 star	Spam	0.983	n/a (spam)

[Insert screenshots of the Analyzer page (one spam, one genuine example) and the Dashboard page from your deployed URL here.]

10.2 Running locally

```
pip install -r requirements.txt
python run_pipeline.py          # trains, evaluates, saves models/ and outputs/
python webapp/app.py            # http://127.0.0.1:8080
```

11 Google Cloud deployment

Service chosen: Cloud Run. The model is tiny (≈ 1 MB of weights plus a JSON vocabulary), the app is stateless and needs only CPU, so a serverless container is the best fit: no servers to manage, HTTPS URL out of the box, automatic scale-to-zero (so the student coupon is barely consumed) and deployment from source in a single command. Alternatives considered: App Engine (heavier runtime config for the same result), Compute Engine VM (manual patching, always-on cost), Vertex AI endpoints (over-sized for a 6 k-parameter NumPy model).

Deployment steps

1. Request the student coupon with the link in the brief (school e-mail), redeem it, and create a project with the credit attached as billing account.
2. Install the gcloud CLI and log in: `gcloud auth login`.
3. Train the model once locally so that `models/spam_model.npz` exists (the container ships these files).
4. Run `PROJECT_ID=<id> ./deploy_gcp.sh`, which executes the commands below. Cloud Build reads the included `Dockerfile` (python:3.11-slim + Flask + NumPy + gunicorn) and Cloud Run starts it.
5. Open the printed Service URL, test one spam and one genuine review, take screenshots and paste the URL on the cover page.

```
gcloud config set project <your-project-id>
gcloud services enable run.googleapis.com cloudbuild.googleapis.com
artifactregistry.googleapis.com
gcloud run deploy review-intelligence --source . --region asia-south1 \
      --allow-unauthenticated --memory 512Mi --cpu 1 --max-instances 2 --port 8080
```

Item	Status
Dockerfile, gunicorn entry point, requirements-app.txt, deploy_gcp.sh	Provided and included in the zip
Local run of the web app and all endpoints (automated tests)	Executed — 9 tests pass
Actual deployment to the student's Google Cloud project	[[To be performed by the student — needs the personal coupon / project]]
Deployed Service URL	[[https://review-intelligence-xxxxx-el.a.run.app]]
Screenshot of Cloud Run console (service 'Ready')	[[Insert screenshot]]
Screenshot of the app in the browser on the Cloud Run URL	[[Insert screenshot]]

Region **asia-south1 (Mumbai)** is used for low latency to Indian users. To avoid charges after grading, delete the service with `gcloud run services delete review-intelligence --region asia-south1`.

12 Limitations and future work

- **Synthetic data.** The corpus is generated from templates, so its language is far less diverse than real reviews; real-world accuracy will be lower and the hyper-parameter conclusions (e.g. '4 hidden neurons are enough') may change. The first next step is re-running the pipeline on a real Flipkart / Amazon.in / SMS-spam dataset (one command).
- **Bag-of-n-grams features** ignore word order and cannot understand context, sarcasm or code-mixed spelling variants; sub-word embeddings or a sequence model (CNN / LSTM / Transformer) would help.
- **Single split.** Hyper-parameters were chosen on one validation split with 1–3 seeds; k-fold cross-validation would give tighter estimates.

- **Sentiment labels come from star ratings**, which are noisy; the topic detector is rule-based and not part of the neural network.
- **Adversarial setting.** Spammers adapt; the model needs periodic retraining, monitoring and human review for borderline scores. Reviewer-level signals (account age, verified purchase, burst timing) are not available here.
- **Explanations** (occlusion) are approximate; they show sensitivity, not causation.

13 Conclusion

A complete spam / fake-review detection system for Indian e-commerce was designed, implemented from scratch with NumPy, evaluated and packaged for Google Cloud Run. The perceptron baseline reaches F1 93.02% and fails on XOR, motivating the MLP, whose hand-written back-propagation was verified against numerical gradients (error ≈ 1e-10). Binary cross-entropy is the right cost for the sigmoid output — equal to MSE from a normal start but able to escape a confidently-wrong initialisation where MSE stalls at 79.4% accuracy. Among the gradient-descent variants, mini-batch training is the practical default, batch GD is far too slow per epoch, SGD is accurate but ~21× slower, and adaptive optimisers minimise the cost fastest. Six hyper-parameters were tuned; the deployed 4-neuron network attains test accuracy 97.75%, precision 99.55%, recall 89.56%, F1 94.29% with 6,049 parameters. These figures hold for the synthetic dataset used; the codebase is ready to be re-run on real data.

References

- Rosenblatt, F. (1958). The perceptron: a probabilistic model for information storage and organization in the brain. Psychological Review, 65(6).
- Minsky, M. & Papert, S. (1969). Perceptrons. MIT Press.
- Rumelhart, D., Hinton, G. & Williams, R. (1986). Learning representations by back-propagating errors. Nature, 323.
- Nesterov, Y. (1983). A method for solving the convex programming problem with convergence rate $O(1/k^2)$. Soviet Math. Dokl.
- Duchi, J., Hazan, E. & Singer, Y. (2011). Adaptive subgradient methods for online learning and stochastic optimization (AdaGrad). JMLR, 12.
- Tieleman, T. & Hinton, G. (2012). Lecture 6.5 – RMSProp. COURSERA: Neural Networks for Machine Learning.
- Kingma, D. & Ba, J. (2015). Adam: a method for stochastic optimization. ICLR.
- Sutskever, I. et al. (2013). On the importance of initialization and momentum in deep learning. ICML.
- He, K. et al. (2015). Delving deep into rectifiers (He initialisation). ICCV. · Glorot, X. & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. AISTATS.
- Goodfellow, I., Bengio, Y. & Courville, A. (2016). Deep Learning. MIT Press (Chapters 6, 8).
- Ott, M. et al. (2011). Finding deceptive opinion spam by any stretch of the imagination. ACL. (background on fake-review detection)

Appendix A How to run the project step by step

1. Unzip `ecom_review_intelligence.zip` and open a terminal in the folder.
2. (Recommended) create a virtual environment: `python -m venv .venv` and activate it (`source .venv/bin/activate`, Windows: `.venv\Scripts\activate`).
3. Install packages: `pip install -r requirements.txt`.
4. Train and run all experiments: `python run_pipeline.py` (≈ 5–10 min; add `--quick` for a 1-minute test). It prints each stage and writes `models/`, `outputs/figures/`, `outputs/results.json`, `outputs/tables/*.csv`.
5. Optional self-test: `python -m unittest discover -s tests -v` (9 tests: gradient check, optimisers, perceptron/XOR, web endpoints).
6. Build this report: `python report/build_report.py` → `report/Review_Intelligence_Report.docx`.
7. Start the web app: `python webapp/app.py` and open http://127.0.0.1:8080.
8. Deploy: follow Section 11 (`PROJECT_ID=<id> ./deploy_gcp.sh`).
9. Use real data: `python run_pipeline.py --data my_reviews.csv` (columns `review_text`, `is_spam`; optional `rating, category, product_id, price_inr, review_date`), then rebuild the report.

Shortcut: `./run_all.sh` (Linux/macOS) or `run_all.bat` (Windows) performs steps 2–7 automatically.

Appendix B Submission checklist

Item	Done?
Student name, roll number and deployed URL filled on the cover page	[[ ]]
Google Cloud coupon requested and redeemed; coupon e-mail saved as PDF / screenshot (required by the brief)	[[ ]]
Application deployed on Cloud Run; screenshots inserted in Sections 10 and 11	[[ ]]
Report uploaded (this document, or exported to PDF)	[[ ]]
Source-code zip uploaded (if requested)	[[ ]]
Brief's deadline: 20 September 2026 — confirm with the course instructor if a late submission is acceptable	[[ ]]